可重現性與超參數步驟：隨機種子、local_epochs 與 lr

進入 FedProx 之前要做什麼、在哪裡修改

0. 目標、原則與檔案

兩個目標。第一，讓每一次實驗都可以重現：相同的設定搭配相同的種子，必須得到相同的數字，如此兩組設定之間的差異才值得信賴。第二，先後了解 local_epochs 與 lr 如何影響全域模型，並透過有控制、分階段的研究選出數值。本指南假設前一份檢查報告中的修正（伺服器傳送 fit 設定、保存每一輪歷史紀錄、彙整 F1 與 AUC）已經完成。本指南只提供步驟與每項修改的位置，程式碼由學生自行撰寫。

四項工作原則

1. 一次只改一件事。先固定 lr 掃描 local_epochs，再固定最佳的 local_epochs 掃描 lr。
2. 以驗證集調整參數，絕不使用測試集。測試集只在最後使用一次。
3. 每一組設定至少使用 5 個種子，比較平均值與標準差，而不是單次執行的結果。
4. 記錄所有內容：設定、種子、套件版本、每一輪指標、各客戶端指標。

需要建立或修改的檔案

檔案	動作
~~src/seed.py~~	~~新增。步驟 1。~~
~~src/fl/data.py~~	~~新增。將 build_client_assets 從 simulation.py 移至此處並擴充。步驟 2。~~
~~src/fl/server.py~~	~~修改。將模組層級的 strategy 改為函式，並擴充策略類別。步驟 3。~~
~~src/fl/client.py~~	~~修改。建構函式、fit 與 evaluate。步驟 4。~~
~~run_experiment.py~~	~~新增。取代 simulation.py 的主程式區塊。步驟 5。~~
check_repro.py	新增。步驟 6。
sweep.py, summarize.py	新增。第二部分。
final_compare.py	新增。取代目前位於 simulation.py 底部的比較程式碼。第二部分。

前提： 尚未看過學生的 src/train_utils.py 與模型檔案。以下步驟假設 train(model, dataloader, epochs, lr) 會依據 lr 建立最佳化器，並只迭代傳入的 DataLoader。若 train() 自行建立 DataLoader 或使用自己的隨機呼叫，請參考步驟 7 的疑難排解表。

第一部分：讓實驗可以重現

1.1 本專案中隨機性的來源

只在腳本開頭設定一個種子是不夠的。下表每一列都是獨立的隨機性來源，各自都需要控制。

來源	位置	控制方式
初始全域權重	伺服器	建立一個有種子的模型，將其權重作為初始參數傳給策略
小批次資料打亂順序	各客戶端的 DataLoader	替 loader 配置有種子的產生器，並在每次 fit 呼叫時重設
Dropout（若有使用）	訓練中的模型	每次 fit 開始時設定 torch 全域種子
訓練／驗證／測試切分	前處理與資料模組	使用固定不變、且與執行種子無關的 random_state
僅本地與個人化模型	最終比較腳本	每次訓練前，依執行種子與客戶端 id 設定種子
聚合順序	伺服器	客戶端完成順序不固定，而浮點數加總與順序有關，因此依客戶端 id 排序結果
平行的 Ray 工作程序	模擬環境	在客戶端程式碼內設定種子，因為腳本開頭設定的種子不會傳到工作程序
GPU 的非確定性	CUDA 運算核心	這類小型表格資料模型建議使用 CPU

步驟 1. 建立統一的種子設定函式

位置：新檔案 src/seed.py。

1. 撰寫一個接收整數種子的函式。
2. 在函式內設定 Python 的 random 模組、NumPy 與 PyTorch（含 CUDA 產生器）的種子。
3. 同時將 PyTorch 的 cuDNN 選項設為確定性模式，並關閉 benchmark。
4. 在 run_experiment.py、check_repro.py 與 final_compare.py 的開頭呼叫它。

步驟 2. 加入固定的驗證集切分

位置：將 build_client_assets 從 simulation.py 移到新檔案 src/fl/data.py，再進行修改。目前的程式碼在每一輪結束後都使用測試集評估。若依據這些數字選擇 local_epochs 或 lr，最終結果會被高估，因此每一輪的評估必須改用驗證集。

1. 定義一個切分種子常數（例如 SPLIT_SEED），它絕不能改變，也不能依賴執行種子。
2. 讀入各組的 train parquet 檔後，以該常數作為 random_state，依標籤 DIQ010 分層，切成訓練與驗證兩部分（約 85% 與 15%）。
3. 測試用的 parquet 檔保持不動。
4. 建立訓練用 DataLoader 時配置自己的 torch.Generator，並讓它可從 loader 取得，以便稍後在客戶端內重設種子。

5. 讓函式回傳四項：模型、訓練 loader、驗證 loader、測試 loader，並更新所有呼叫它的地方。

保持檔案固定： 若原本的前處理使用 train_test_split 但未設定 random_state，請先修正，重新產生一次 parquet 檔，之後保持不變。

步驟 3. 伺服器：有種子的起點、設定與固定順序聚合

位置：src/fl/server.py。

1. 將模組層級的 strategy 物件改為函式（例如 make_strategy），接收 seed、input_dim、local_epochs、lr 以及全域模型檔的輸出路徑。所有匯入 strategy 的地方都必須改為呼叫此函式。
2. 在函式內設定 torch 種子、建立一個 DiabetesNet、將其權重轉成 Flower 參數，並作為策略的初始參數傳入。這會取代目前由 Flower 向隨機客戶端索取隨機權重的行為。
3. 定義一個 fit 設定函式，回傳 local_epochs、lr、回合數與種子，並將其傳給策略作為 fit 設定函式。回合數也讓之後使用學習率排程成為可能。
4. 擴充 weighted_average，使其彙整準確率、F1 與 AUC，而不只是準確率。
5. 在 SaveModelStrategy 中，以建構函式參數取得模型檔路徑，取代模組常數，使每次執行都寫入自己的資料夾，各次執行之間不會互相覆寫。
6. 在 aggregate_fit 中，先依客戶端 id 排序結果再呼叫父類別方法。客戶端 id 來自 fit 回傳的指標（見步驟 4）。aggregate_evaluate 也做相同處理。
7. 在 aggregate_evaluate 中，另外將各客戶端的回合數、客戶端 id、loss、準確率、F1 與 AUC 加入策略上保存的清單。執行結束後，將此清單存成各客戶端的紀錄。

步驟 4. 客戶端：不使用預設值、每次呼叫都重設種子

位置：src/fl/client.py。

1. 修改建構函式，使其同時接收客戶端 id，並接收驗證 loader 而非測試 loader。將 id 以整數保存。
2. 在 fit 中直接從 config 讀取 local_epochs 與 lr，不再使用附預設值的 get。舊的預設值掩蓋了伺服器從未傳送這些設定的事實；直接讀取能讓缺少設定時明確報錯。
3. 在 fit 開頭，由執行種子、回合數與客戶端 id 計算出呼叫種子（例如 種子 加 1000 乘以回合數 加 id）。用它設定 torch 種子，並重設訓練 loader 的產生器種子。這必須在 fit 內完成，因為 Flower 每次呼叫都會重新建立客戶端。
4. 讓 fit 在指標字典中回傳客戶端 id，因為伺服器要用它排序。
5. 在 evaluate 中使用驗證 loader。將指標轉為一般 Python 浮點數，並加入客戶端 id。

步驟 5. 一次執行 = 一次腳本呼叫

位置：專案根目錄的新檔案 run_experiment.py，取代 simulation.py 的主程式區塊。請從專案根目錄執行，因為資料路徑是相對路徑。

1. 接受命令列參數：seed、local_epochs、lr、rounds 與輸出資料夾。
2. 呼叫種子設定函式、由某個客戶端的資料取得輸入維度、以 make_strategy 建立策略，並以本檔案定義的客戶端函式呼叫 start_simulation（它以 build_client_assets 與新的建構函式建立客戶端）。

3. 執行結束後，在輸出資料夾寫入三項內容：策略中的各客戶端紀錄、含所有參數與 Flower、PyTorch、Python 版本的設定檔，以及取自 start_simulation 所回傳 History 物件的每一輪歷史（回合、驗證 loss、準確率、F1、AUC）。
4. 最後才寫入歷史檔，讓它的存在代表該次執行已完成。
5. 每次執行都要以獨立的程序啟動。在同一個 Python 程序中重複呼叫 start_simulation 可能使 Ray 處於不乾淨的狀態，並悄悄破壞可重現性。

步驟 6. 證明實驗可以重現

位置：新檔案 check_repro.py。不要假設種子設定一定有效。

1. 以 3 回合啟動 run_experiment.py 三次：種子 0 兩次（輸出到不同資料夾）、種子 1 一次。
2. 從各資料夾載入儲存的最終全域權重，計算兩次種子 0 之間，以及種子 0 與種子 1 之間的最大絕對差異。
3. 逐一比較兩次種子 0 的歷史檔數值。

結果	意義
相同種子：差異為 0 或小於 1e-6；歷史紀錄到小數第 4 位相同	可重現，可進入第二部分。
不同種子：差異明顯較大	確認種子確實會改變執行結果。若也是 0，表示種子沒有傳入程式碼。
相同種子但差異達 1e-3 以上	無法重現，請參考下方疑難排解表。

步驟 7. 相同種子卻得到不同數字時的疑難排解

可能原因	檢查或修正方式
train() 自行建立 DataLoader 或自行打亂資料	打開 src/train_utils.py，確認它只迭代傳入的 loader，且沒有其他隨機呼叫。
Dropout 或其他隨機層	已由 fit 內設定 torch 種子涵蓋，請確認 fit 確實有做。
DataLoader 的 num_workers 大於 0	使用 num_workers=0（預設值），工作程序會帶來額外的隨機性。
前處理中切分時未設定 random_state	修正後重新產生一次 parquet 檔，並保持檔案不再變動。
聚合順序	確認兩個 aggregate 方法都有排序結果，且 fit 有回傳客戶端 id。
使用 GPU	調參研究改用 CPU，或啟用 PyTorch 的確定性演算法選項。
先前當機遺留的 Ray 程序	停止 Ray（ray stop）後再重做檢查。

合理的期望： 目標是權重逐位元完全相同，但在某些機器上仍可能殘留約 1e-7 的差異。權重差異小於 1e-6 且各指標到小數第 4 位相同，即視為可重現。步驟 3 的排序是依據 Flower 收集結果的方式所做的設計，請以步驟 6 驗證，而非直接假設有效。

第二部分：了解並調整 local_epochs 與 lr

2.1 研究的基本規則

項目	設定
選擇指標	全域模型在最後一輪的平均驗證 AUC；F1 與準確率為次要指標。
種子	調參使用 0、1、2、3、4，每組設定都相同，因此可以做成對比較。
回合數	掃描使用 20 回合（目前的 10 回合可能太短，看不出小數值的收斂情況）。
調參期間固定不變	模型、資料、批次大小、客戶端數量、FedAvg。僅本地與個人化的設定不在此處調整。
使用的資料	僅使用驗證集，不使用測試資料。

2.2 各設定的作用

- local_epochs 是每個客戶端在一輪內對自己的訓練資料進行的訓練週期數。週期越多，每一輪的進展越大，但各客戶端也越偏向自己的資料。在非獨立同分佈（non-IID）的分區下，這些偏離後的模型平均結果，可能比溫和更新後的平均結果更差（客戶端漂移）。
- lr 是每次更新的步長。太小則學習緩慢，20 回合可能不足；太大則訓練雜亂甚至發散。
- 兩者會互相影響：合起來決定客戶端在一輪內移動多遠。因此要分階段調整，最後再用小範圍的組合加以確認。
- lr 的意義取決於 train() 內的最佳化器。請確認是 SGD 還是 Adam 並寫入報告。純 SGD 常見的好數值約為 0.01 至 0.1，Adam 約為 0.001 至 0.01，請依實際使用的最佳化器選擇掃描範圍。

階段 A0. 冒煙測試：設定是否真的傳到客戶端？

原本的程式碼從未傳送 local_epochs，因此更動它不會有任何效果。掃描之前，請以相同種子分別用 local_epochs 為 1 與 5 執行 run_experiment.py 2 回合，確認驗證指標不同。若完全相同，表示該設定沒有被使用。

掃描所需的工具腳本

位置：專案根目錄的兩個新檔案。

1. sweep.py：接收階段名稱，並對組合與種子進行迴圈。每個組合都以獨立程序啟動 run_experiment.py，並寫入以階段、local_epochs、lr 與種子命名的資料夾（例如 results/stageA/E2_lr0.01/seed3）。若某次執行的歷史檔已存在就略過，使中斷的掃描可以直接重新啟動。需要四個階段：stageA、stageB（以最佳 local_epochs 為參數）、stageC（接收 local_epochs 與 lr 的清單）以及 final（接收選定的 local_epochs 與 lr，並使用種子 10 至 14）。
2. summarize.py：接收階段名稱，讀取其下所有歷史檔，並印出各組設定的最終回合 AUC、F1 與準確率的平均值與標準差。它也計算達到總 AUC 進步量 95% 所需的回合數（以平均曲線，由第 1 回合到最後一回合計算），並儲存兩張含標準差帶的平均驗證 AUC 圖：一張對回合數，一張對總本地訓練週期數（回合數乘以 local_epochs）。local_epochs 的值可由資料夾名稱取得。

3. 成對比較：由於每組設定使用相同的種子，請逐種子比較各組設定（相同種子有相同的初始權重）。對每個種子，以一組設定的最終 AUC 減去另一組，再看這些差值的平均與標準差，這能去除大部分雜訊。

階段 A. 掃描 local_epochs（lr 固定為 0.01）

1. 執行階段 A：local_epochs 為 {1, 2, 3, 5, 10}，lr = 0.01，20 回合，種子 0 至 4，共 25 次執行。
2. 對該階段執行 summarize.py 並查看兩張圖。對回合數的圖顯示每一輪的成本；對總訓練週期的圖顯示更多本地訓練是否真的更有效率，還是只是做得更多。
3. 依下表解讀結果，再套用決策規則。

現象	可能意義	行動
較大的 local_epochs 上升較快且最終較高	更多本地訓練有幫助，漂移尚不是問題	嘗試更大的值，或在相同總訓練週期下比較
較大的 local_epochs 上升較快，但平台較低或各回合間波動	non-IID 分區造成的客戶端漂移	選擇較小的值；這也是之後使用 FedProx 的動機。
所有數值的最終結果相差不到一個標準差	此設定對最終品質影響不大	選擇穩定且最小或成本最低的值
不同數值的曲線完全相同	設定沒有傳到客戶端	重做階段 A0 並檢查 fit 設定函式
隨 local_epochs 增加，某一組的 AUC 下降而其他組上升	該分區與其他分區差異最大	檢視各客戶端紀錄並寫入報告

決策規則： 選擇最小的 local_epochs，條件是其最終平均驗證 AUC 與最佳值相差不超過一個標準差（跨種子），且曲線平滑。記為 E*，並寫下選擇與理由。

階段 B. 掃描 lr（local_epochs 固定為 E*）

1. 執行階段 B：lr 為 {0.001, 0.003, 0.01, 0.03, 0.1}，local_epochs = E*，20 回合，種子 0 至 4，共另外 25 次執行。數值在對數尺度上等距排列，因為 lr 的影響是倍數式的。
2. 執行 summarize.py 並依下表解讀結果。
3. 若最佳值落在組合的邊緣，請先朝該方向擴大範圍並重複執行，再做決定。
4. 套用相同的一個標準差規則選出 lr*。

現象	可能意義	行動
緩慢穩定上升；第 20 回合仍在上升	lr 太小	使用較大的 lr 或增加回合數
平滑上升到平台	良好範圍	依一個標準差規則選擇最佳值
曲線雜亂、標準差帶很寬、AUC 在回合之間下降	lr 太大	使用較小的 lr
驗證 loss 變成 nan 或非常大	訓練發散	排除該數值並於報告中註明

找到最佳固定 lr 之後的選擇性延伸：遞減排程，前期步長大、後期步長小。位置：make_strategy 內的 fit 設定函式。將基準 lr 乘以衰減係數（例如 0.9）的「回合數減一」次方。請將它作為獨

立的設定，與最佳固定 lr 比較。

階段 C（建議）. 確認交互作用

階段 A 與 B 一次只改一個設定，可能會錯過更好的組合。請在勝出的數值周圍以小範圍組合執行階段 C：例如 E* 周圍的三個 local_epochs 值，搭配 lr* 周圍的三個 lr 值，共 9 種組合乘以 5 個種子。若鄰近的組合比階段 A、B 的選擇好超過一個標準差，就採用它。

最終確認：全新種子與測試集

選定（E*, lr*）後，請以調參時沒有使用過的種子（10 至 14）執行 final 階段。以種子 0 至 4 挑出勝出者，會讓這些種子的結果略微偏好，因此使用全新種子才能得到誠實的估計。接著撰寫 final_compare.py，這是唯一會使用測試集的腳本。位置：專案根目錄的新檔案，取代 simulation.py 底部的比較程式碼。

1. 對種子 10 至 14 的每一個種子與五個群組中的每一組，載入該種子儲存的最終全域權重，以及 build_client_assets 提供的該組 loader。
2. 每次訓練之前，依種子與群組 id 設定所有種子，並重設訓練 loader 的產生器種子。
3. 僅本地：全新的模型，以與各 FL 客戶端相同的總訓練量（回合數乘以 E* 個訓練週期，使用 lr*）訓練。這能讓比較公平。
4. 僅全域：載入全域權重後直接評估。
5. 個人化：以目前的個人化設定（3 個訓練週期、0.005）微調全域權重；這組設定在下一份指南之前維持不變。
6. 在該組的測試集上評估三種模型，將每一列結果存檔，並印出各群組與各模型跨種子的平均值與標準差。

第三部分：交付項目與檢查清單

學生應交付的內容

- 可重現性證據：check_repro.py 的輸出（相同種子、不同種子）。
- 表 A：各 local_epochs 數值的最終驗證 AUC、F1 與準確率（5 個種子的平均值與標準差），以及達到 95% 進步量所需的回合數。
- 圖 A：兩張曲線圖（對回合數與對總訓練週期），含標準差帶。
- 表 B 與圖 B：lr 掃描的相同內容；若執行了階段 C，則加上表 C。
- 決策紀錄：選定的 E* 與 lr*、使用的規則、理由、日期，以及設定檔中的環境版本。
- 最終表格：各組在測試集上的僅本地、僅全域與個人化結果（種子 10 至 14 的平均值與標準差）。

檢查清單

步驟	完成條件
1	種子設定函式已建立，且在每個腳本開頭呼叫
2	驗證集以專屬常數固定；每一輪評估使用驗證集而非測試集
3	make_strategy 傳入有種子的初始權重與 fit 設定；兩個 aggregate 方法都依客戶端 id 排序
4	客戶端沒有設定預設值，且在 fit 內重設種子
6	check_repro.py 通過：相同種子一致、不同種子不同
A0	冒煙測試顯示 local_epochs 1 與 5 的結果不同
A	25 次執行完成；依一個標準差規則選出 E*
B	25 次執行完成；選出 lr*；最佳值不在組合邊緣
C	已檢查鄰近組合（建議）
Final	全新種子執行與 final_compare.py 完成；測試集僅在此使用

下一步： 有了可重現的基準以及調好的 (E*, lr*) 之後，就能以相同的種子與相同的驗證流程，公平地比較 FedProx 與一般的 FedAvg。接著自然的後續研究是個人化的訓練週期與學習率，以及使用提早停止的僅本地基準。